Intermediate (real branching) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `conditional_routing_practice.py`: a plain `add_edge` always goes to the same next node — but a real agent needs to *decide* where to go next, based on what just happened. `add_conditional_edges` is that decision point: it takes a small routing function that inspects the state and returns a key, which picks the next node from a mapping you supply. This is the one piece of machinery every real agent graph, including Doc07's ReAct loop rebuilt as a graph, actually needs. **If not:** `agent_loop_to_graph` would be the first place you ever wired a decision into a graph, with no smaller example to check it against.

Solution

```
# conditional_routing_practice.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class GraphState(TypedDict):
    flag: bool
    message: str

def node_decide(state: GraphState) -> dict:
    return {}

def node_a(state: GraphState) -> dict:
    return {"message": "went down path A"}

def node_b(state: GraphState) -> dict:
    return {"message": "went down path B"}

def route_on_flag(state: GraphState) -> str:
    if state["flag"]:
        return "path_a"
    return "path_b"

builder = StateGraph(GraphState)
builder.add_node("node_decide", node_decide)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
```

```
builder.add_edge(START, "node_decide")
builder.add_conditional_edges(
    "node_decide", route_on_flag, {"path_a": "node_a", "path_b": "node_b"},
)
builder.add_edge("node_a", END)
builder.add_edge("node_b", END)

graph = builder.compile()

for flag_value in (True, False):
    result = graph.invoke({"flag": flag_value, "message": ""})
    print(f"flag={flag_value} -> {result['message']}")
```

Expected output:

```
flag=True -> went down path A
flag=False -> went down path B
```